

Journey Profile Generator

08/11/2025

Project Description

Vision

The goal of this project is to develop an independent software application that generates structured JSON files containing detailed route and timing information for a train journey specified by the user. These JSON files are called Journey Profiles. The application will enable planners to load a given infrastructure file, enter the train number, select a route, and define timing points. Optionally, the user can specify speed limitations for selected links. Afterwards, they can choose the train type as well as a driving strategy. Finally, they can export the findings as a Journey profile. The long-term vision is a reliable and intuitive tool that can be used by planners or researchers. Generated profiles will be used to showcase projects of the EBD (Eisenbahnbetriebsfeld Darmstadt). The application will be used in a more complicated context, i.e. the generator can handle larger infrastructure files.

The underlying problem is that current processes for generating such profiles are often manual, fragmented, or depend on multiple tools, which makes them less efficient and more prone to errors. A unified application streamlines this workflow.

Current State and Target State

Current state

Currently, example infrastructure files and Journey Profiles exist. Additionally, we have acceleration and deceleration data for different train types. No user interface or automated generation process exists yet.

Target state

By the end of the project, a fully working desktop application with a graphical user interface should be available. Users will be able to load the infrastructure file, visually explore and select relevant links and timing points, and put in arrival and departure times associated with these positions. Furthermore, the application will allow the selection of predefined train types and driving strategies. This information will be used in the computation of the Journey Profiles. Based on the inputs, the system will test whether the given time constraints are feasible, potentially via a quick analysis for timetable verification. If not, the system will generate the best possible solution.

The client emphasizes code quality, specifically clear function segmentation and documentation, to ensure it is easily maintained and extended by other teams in the future.

Domain Description

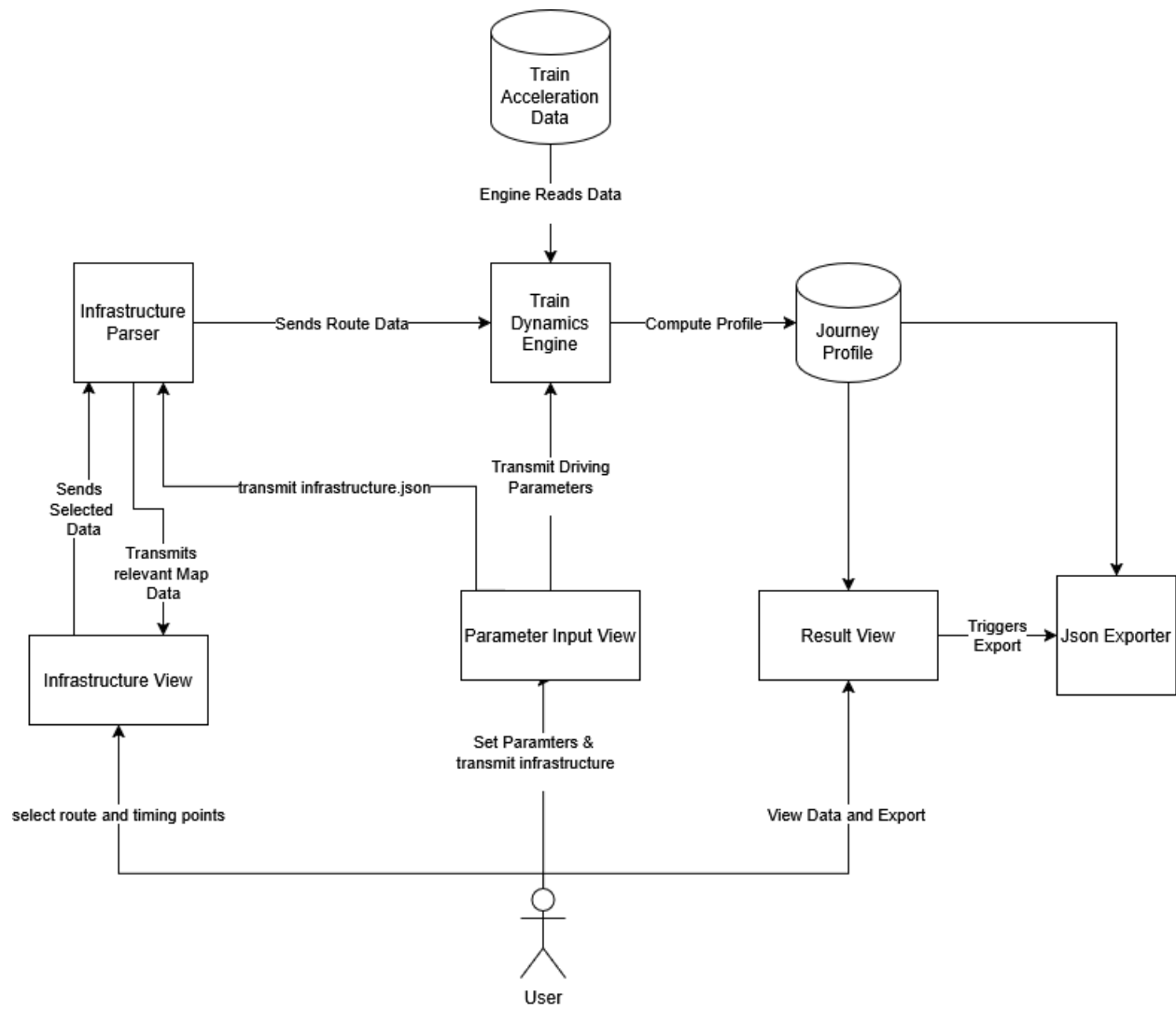
The application operates within the field of railway operations and transportation planning, and belongs to the broader domain of civil engineering. A Journey Profile consists of a timetable accompanied by further information relevant for analysis. It combines timing points, point types (STOP or PASS), and information, whether the train stops at a given point.

Timing points represent specific locations along a route where planners define operational constraints such as fixed arrival, departure times, or other points of interest.

Driving strategies describe how the train should behave dynamically on each segment, such as energy-efficient driving or coasting.

The primary users are railway planners or timetable coordinators, who load infrastructure data, select routes and timing points, enter scheduling information, and generate the final Journey Profile. Indirect stakeholders include timetable users (e.g., passengers who depend on robust timetables) and system maintainers who rely on clarity, maintainability, and extendability.

The application must adhere to the predefined train dynamics and the features of the “infrastructure_layout” JSON file. All generated Journey Profiles must follow the required schema to ensure compatibility in further processing.



Results

1: User Application, Hardware, Environment, Libraries

The project will involve the implementation of a desktop application running on an x86 system (Windows 11, desktop). Development will be carried out in the Python programming language and in the PyCharm development environment using the pySide6 library.

2: Tests

The test data will be provided by the client in the form of input JSON files for the infrastructure. Additional files will be created as needed. In addition, unit tests will be developed and used.

3: (Delivery of) Artifacts

The finished artifacts include the program or its program code with the associated documentation and the developed tests, as well as their test results. The aforementioned artifacts will be delivered via the code-sharing platform GitLab.

4: Change Requests

Change requests will be discussed with the client, depending on their feasibility, and taken into account accordingly.

5: Minimum Scope

The software should dynamically create and display the associated infrastructure based on a file that has been read in and data entered manually by the user. The data should be used to perform calculations and output the appropriate journey (or the best option).

Risk Factors

1: A Team Member is unable to work for an extended period due to illness

We believe the likelihood of this scenario occurring to be around 10%. In this case, the workload for all other team members would increase substantially, especially if they had to complete a part of the project that only the sick person truly understands. Moreover, since there is a time constraint, the quality of the final product may be impacted.

Our team has thought up several mitigation strategies: First of all, prevention is the best cure, so we will try to keep a healthy lifestyle. Second of all, in case of sickness, the sick team member should explain the inner workings of their parts of the project to the rest of the group, so they can take over the reins. Finally, the sick team member will resume their work from home as early as possible. This way, the impact on the quality of work should be mostly mitigated.

2: Needed Resources suddenly become unavailable

For this scenario, we estimate a likelihood of about 5%. Our Project mostly relies on GitLab, which has a great track record. We don't need access to special laboratories or administrators. If GitLab had a temporary outage, we would lose access to our synchronization software. It would be harder for our client to track our progress, and for the team members to build upon other members' code.

However, working around this scenario would not be particularly difficult. The team could simply pivot to GitHub, as long as the client agrees. Otherwise, copy and pasting code between members and the client would also be an option.

3: The Client is unhappy with the Group's performance

Based on our estimation, the probability of this scenario is quite low. Our Group has known each other for a while, and all members are in agreement about striving to deliver the best possible results. However, if the client was unhappy with our performance, it could seriously impact group morale and thus even further erode the quality of our work.

To mitigate this risk, we plan to keep our client as involved as possible in our progress by hosting (at least) biweekly progress update meetings and inquiring about his opinion on the direction of the project. If the client voices unhappiness, we will try our best to take his feedback into consideration and adjust course.

Legal

Legal Information

The project is open source and will be published on GitHub under an MIT license. There is no confidentiality agreement between the developers and the client. Furthermore, the students do not plan to continue using the project themselves at a later date.

The students expect the client to hold meetings every two weeks to clarify questions, problems, and misunderstandings and to communicate interim results. In addition, the students expect additional explanations of the provided sample files for a deeper understanding. Furthermore, regular feedback and open communication (via email) are desired.

The team can be reached at the email address lasse.reith@stud.tu-darmstadt.de. Inquiries will be forwarded directly to the group, discussed internally, and answered within three business days at the latest.